

Konkurentno programiranje (nastavak)

Katedra za automatsko upravljanje
Fakultet tehničkih nauka
Novi Sad

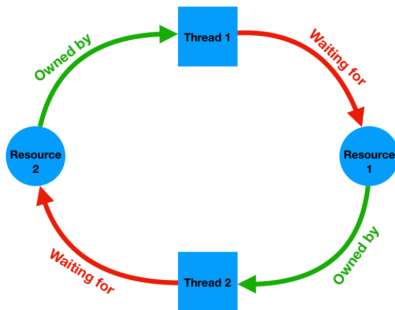
31. mart 2026.

- sinhronizacija: odustajanje od konkurentnosti da bi se program izvršavao kako treba
- najčešći način sinhronizacije je korišćenjem *mutex*-a (***mutually exclude***) koji samo jednoj niti omogućava da uđe u kritičnu sekciju za koju je on zadužen (na primer čitanje i pisanje u jednu promenljivu)
- *lock* i *unlock* označavaju početak i kraj kritične sekcije, u C#-u je *lock* ograničen blokom koda
- kada druga nit naiđe na zaključanu sekciju ona se blokira, što izaziva *context switch*
- **kritična sekcija bi trebalo da bude što kraća** (želimo što više koda da se dešava konkurentno)

Deadlock

Pojava da nit 1 čeka na nit 2 a nit 2 čeka na nit 1. Ova situacija ne izaziva nikakvu grešku ali će deo ili ceo program biti blokiran.

Izbegavati ugnježdene *lock-ove*! Ukoliko su nužni neka budu u istom redosledu, ili jednostavno koristiti *scoped lock*.



<https://deadlockempire.github.io/>

Deadlock - primer

Da li će doći do deadlock-a?

```
object alpha = new object();
object beta = new object();
object gamma = new object();
```

```
// thread 1
lock (alpha)
{
    // do stuff 1
}

lock (gamma)
{
    lock (beta) // <===
    {
        // do stuff 2
    }
}

/* finished */
```

```
// thread 2
lock (gamma)
{
    lock (alpha)
    {
        lock (beta)
        {
            // do stuff 3
        }
    }
}

/* finished */ // <===
```

```
// thread 3
lock (gamma)
{
    lock (alpha)
    {
        // do stuff 4
    }
}

lock (beta)
{
    lock (gamma) // <===
    {
        // do stuff 5
    }
}

/* finished */
```

Deadlock - primer

Da li će doći do deadlock-a?

```
object alpha = new object();
object beta = new object();
object gamma = new object();
```

```
// thread 1
lock (alpha)
{
    // do stuff 1
}

lock (gamma)
{
    lock (beta) // <===
    {
        // do stuff 2
    }
}
```

```
/* finished */
```

```
// thread 2
lock (gamma)
{
    lock (alpha)
    {
        lock (beta)
        {
            // do stuff 3
        }
    }
}

/* finished */ // <===
```

```
// thread 3
lock (gamma)
{
    lock (alpha)
    {
        // do stuff 4
    }
}

lock (beta)
{
    lock (gamma) // <===
    {
        // do stuff 5
    }
}

/* finished */
```

Da.

Deadlock - primer

Da li će doći do deadlock-a?

```
object alpha = new object();
object beta = new object();
object gamma = new object();
```

```
// thread 1
lock (alpha)
{
    // do stuff 1 <===
}
```

```
lock (gamma)
{
    lock (beta)
    {
        // do stuff 2
    }
}
```

```
/* finished */
```

```
// thread 2
lock (gamma)
{
    lock (alpha) // <===
    {
        lock (beta)
        {
            // do stuff 3
        }
    }
}

/* finished */
```

```
// thread 3
```

```
lock (gamma)
{
    lock (alpha)
    {
        // do stuff 4
    }
}
```

```
lock (beta)
{
    lock (gamma)
    {
        // do stuff 5
    }
}
```

```
/* finished */    // <===
```

Deadlock - primer

Da li će doći do deadlock-a?

```
object alpha = new object();
object beta = new object();
object gamma = new object();
```

```
// thread 1
lock (alpha)
{
    // do stuff 1 <===
}
```

```
lock (gamma)
{
    lock (beta)
    {
        // do stuff 2
    }
}
```

```
/* finished */
```

```
// thread 2
lock (gamma)
{
    lock (alpha) // <===
    {
        lock (beta)
        {
            // do stuff 3
        }
    }
}

/* finished */
```

```
// thread 3
```

```
lock (gamma)
{
    lock (alpha)
    {
        // do stuff 4
    }
}
```

```
lock (beta)
{
    lock (gamma)
    {
        // do stuff 5
    }
}
```

```
/* finished */    // <===
```

Ne.

Ostale sinhronizacijske primitive

- *Spinlock*
 - kad nit želi da uđe u kritičnu sekciju pokušava da *set*-uje *flag* i ako je već *set*-ovan čeka, kad izađe resetuje *flag*
 - *busy waiting* niti, bez *context switching*-a ali *CPU* se ne oslobađa
 - kernel ga koristi, preporuka je da se izbegava u korisničkom prostoru (eng. *user-space*), problem kad se nit koja drži *spinlock* preključi
 - *Task* u *C#*-u minimizuje preključivanje, prvo će čekati malo
- *Condition variable*
 - blokiranje niti dok se ne ispuni neki uslov
 - pošto nit koja čeka proverava uslov (*read*) koji će neka druga nit ispuniti (*write*), taj uslov mora biti pod *mutex*-om pa se *condition variable* uvek koriste uz *mutex*
 - nema ih u *C#*-u direktno, može preko *Monitor* klase ili semafora
 - primer implementacije je na sledećem slajdu
- Semafor - ograničava broj niti koji će pristupiti resursu, sadrži brojač koji se dekrementira kad neko pristupi resursu a inkrementira kada ga otpusti
- Barijera - čeka da sve niti dođu u jednu tačku pa prelazi u drugu fazu

Condition variable primer

```
class Buffer
{
    private readonly object lockObject = new object();
    private bool dataReady = false;

    // Consumer: Waits until data is ready
    public void Consume()
    {
        lock (lockObject)
        {
            while (!dataReady)
            {
                // Atomically releases the lock and waits.
                // Reacquires the lock before returning.
                Monitor.Wait(lockObject);
            }
            Console.WriteLine("Data consumed.");
            dataReady = false;
        }
    }

    // Producer: Prepares data and notifies the consumer
    public void Produce()
    {
        lock (lockObject)
        {
            // ... prepare data ...
            dataReady = true;
            Console.WriteLine("Data produced. Notifying consumer...");
            Monitor.Pulse(lockObject); // Notify one waiting thread
        }
    }
}
```